

PizzaShop PWA - Enunciado do Projeto

Projeto Final — PizzaShop PWA

Plataforma White-Label de Venda de Pizzas Online

1. Contexto

Sua equipe foi contratada para desenvolver um **produto SaaS**: uma Progressive Web App (PWA) de venda de pizzas online que possa ser **licenciada e customizada para diferentes pizzarias clientes**. Ou seja, vocês não vão construir “o site de uma pizzaria”, e sim **um motor de e-commerce de pizzas reconfigurável**, onde cada cliente (pizzaria) pode alterar cores, logotipo, cardápio, taxas de entrega e formas de pagamento sem que uma única linha de código precise ser alterada.

A aplicação deve cobrir o **ciclo completo de compra**: apresentação da marca → cardápio → montagem do pedido → carrinho → checkout → pagamento → acompanhamento da entrega.

2. Objetivos de Aprendizagem

Ao final do projeto, o grupo deve ser capaz de demonstrar domínio em:

- Arquitetura de aplicações React modernas e escaláveis
 - Consumo de APIs REST com fetch, tratamento de estados de carregamento/erro e cache
 - Construção de uma PWA instalável, com funcionamento offline básico
 - Design de sistemas configuráveis/multi-tenant (white-label)
 - Gerenciamento de estado global e local de forma organizada
 - Boas práticas de componentização, tipagem e testes
-

3. Stack Técnica Obrigatória

Camada	Tecnologia sugerida
Build tool	Vite
Linguagem	TypeScript (obrigatório)
UI	React 18+ (function components + Hooks)
Roteamento	React Router (rotas separadas para área do cliente e área administrativa)
Estado global	Context API + useReducer ou Zustand
	CSS Modules, Tailwind CSS ou

Estilização	Styled Components — desde que suporte theming dinâmico
Requisições HTTP	fetch nativo (proibido usar Axios, para forçar o domínio da API nativa)
PWA	Web App Manifest + Service Worker (Workbox ou manual)
Formulários	React Hook Form + Zod (validação)
Testes	Vitest + React Testing Library (mínimo de cobertura definido pelo professor)
Persistência local	localStorage/IndexedDB para carrinho e cache offline

A API de pizzas pode ser uma API pública (ex: [PizzaAPI/pizza-hunt-esque mocks], JSONPlaceholder adaptado) **ou** uma API fake construída pelo próprio grupo com json-server/MSW. O importante é que o consumo seja feito 100% via fetch, com endpoints REST reais (GET, POST, PUT, DELETE).

4. Requisitos Funcionais

4.1 Modo Cliente (loja da pizzaria)

- Landing / Apresentação**
 - Exibe logotipo, cores e nome da pizzaria configurados no módulo admin
 - Seção institucional (sobre, horário de funcionamento, endereço)
- Cardápio**
 - Lista de pizzas consumida via fetch de uma API
 - Filtros (categoria, ingredientes, tamanho) e busca
 - Estados de loading, erro e “sem resultados” tratados visualmente
- Detalhe do produto**
 - Escolha de tamanho, borda, ingredientes extras, observações
- Carrinho**
 - Adicionar, remover, alterar quantidade
 - Cálculo dinâmico de subtotal, taxa de entrega e total
 - Persistência do carrinho (sobrevive a reload da página)
- Checkout**
 - Dados do cliente e endereço de entrega
 - Seleção de forma de pagamento (deve respeitar as opções habilitadas no módulo admin — ex: pix, cartão, dinheiro)
 - Validação de formulário completa
- Pagamento**
 - Simulação de gateway de pagamento (não é necessário integrar um gateway real, mas o fluxo deve ser realista: tela de processamento, sucesso, falha)
- Acompanhamento do pedido / entrega**
 - Tela de status do pedido (recebido → preparo → saiu para entrega → entregue)
 - Deve ser possível simular a mudança de status (via admin ou timer) e refletir isso na tela do cliente

4.2 Modo Administrativo (painel da pizzaria)

- Autenticação** simples (mock de login) para acessar o painel
- Módulo de Customização (o diferencial do projeto)**

- Upload/URL do logotipo
 - Definição de cor primária/secundária (aplicada dinamicamente em toda a loja — theming em tempo real)
 - Nome da pizzeria, endereço, horário de funcionamento
 - Ativar/desativar formas de pagamento aceitas
 - Definir taxa de entrega, raio de entrega, tempo médio de preparo
3. **Gestão de cardápio:** CRUD de pizzas (nome, preço, ingredientes, imagem, categoria)
 4. **Gestão de pedidos:** listar pedidos recebidos e atualizar status (alimenta a tela de acompanhamento do cliente)
 5. **Dashboard simples:** número de pedidos, faturamento do dia/mês (pode ser mockado)

Critério de arquitetura: as configurações do módulo admin devem ser tratadas como uma **fonte única de verdade** (ex: um objeto `tenantConfig` centralizado), consumida por toda a aplicação cliente. Isso simula como o mesmo código serviria a N pizzarias diferentes.

5. Requisitos Não-Funcionais / Boas Práticas Avaliadas

- **Componentização:** componentes pequenos, reutilizáveis e com responsabilidade única
- **Separação de camadas:** `services/` (chamadas de API), `hooks/`, `components/`, `pages/`, `context/` ou `store/`
- **Tratamento de erros** consistente nas chamadas fetch (try/catch, feedback visual ao usuário)
- **Responsividade** (mobile-first, já que é uma PWA)
- **Acessibilidade** básica (uso semântico de HTML, alt em imagens, contraste, navegação por teclado)
- **PWA real:** instalável, ícones, splash screen, funcionamento mínimo offline (ex: cardápio em cache)
- **Performance:** lazy loading de rotas, otimização de imagens
- **Código limpo:** ESLint + Prettier configurados e sem warnings
- **Commits organizados** no Git (histórico com mensagens significativas)

6. Entregáveis

1. Repositório no GitHub com README explicando como rodar o projeto
2. Aplicação publicada (Vercel, Netlify ou similar)
3. Diagrama simples da arquitetura (pode ser feito no Figma, Excalidraw, etc.)
4. Vídeo curto (3-5 min) demonstrando o ciclo completo: cliente comprando e admin customizando a loja em tempo real
5. Relatório técnico curto (1-2 páginas) descrevendo decisões de arquitetura, principalmente do módulo de customização white-label

7. Critérios de Avaliação (sugestão de rubrica)

Critério	Peso
Fluxo completo de compra (apresentação → pagamento →	25%

entrega)	
Módulo de customização/white-label funcional e centralizado	20%
Qualidade do consumo de API via fetch (loading/erro/cache)	15%
Implementação PWA (manifest, service worker, instalável)	15%
Boas práticas de código React/TypeScript	15%
Responsividade, acessibilidade e UX	10%

8. Desafios Bônus (opcional, pontuação extra)

- Multi-idioma (i18n) configurável por pizzeria
 - Modo escuro automático baseado na cor primária configurada
 - Notificações push (Web Push API) avisando mudança de status do pedido
 - Testes E2E com Playwright ou Cypress
 - Deploy multi-tenant real (mesmo código, subdomínios diferentes simulando pizzarias distintas)
-

Dúvidas técnicas podem ser levantadas nos horários de monitoria. Boa sorte, e lembrem-se: o objetivo não é só “fazer funcionar”, é construir algo que poderia, de verdade, ser vendido como produto.